Big Data Project Report

Movie Ticket Booking System

Prepared by:

Wasin Srirachtrakul, 2420110286

Thai-Nichi Institute of Technology

February 2026

1. Introduction

For the project, two main objectives were in mind: First, be able to do the four basic operations of persistent storage: Create, Read, Update, and Delete, or CRUD for short. And second, be able to store the data within MongoDB Atlas.

2. Project Description

A movie ticket booking website where MongoDB is used for storing users, movies, and bookings data.

3. Objective

The objective of this project is to be able to integrate MongoDB into the movie booking website using it as a data storage, as well as to make sure the website runs smoothly with no bugs or any unintentional features.

4. Scope

The project includes movie booking, viewing booking history, and booking deletion system. For future additions, the website will get a new design of the UX and UI, more features for booking, and some quality of life changes.

5. System Design

Backend

database.py

- Serves as a connector for Visual Studio Code and MongoDB Atlas. Using the pymongo module and installing the MongoDB extension in Visual Studio Code, the MongoDB Atlas can be connected to the program. It is an essential file for inserting or pulling data for further uses.

users.py

- Used for creating new users or detecting already registered emails. If the email already exists within the database, the program will not insert it as new data hence fixing data duplication problem.

movies.py

- Used for inserting new movies including information such as title, genre, duration and showtimes. This file also has a way to detect duplicated movies.

bookings.py

- Used for inserting booking data into MongoDB Atlas including information such as user ID, movie ID, showtime, and seat number. It also has a duplicate data detection system in case of an error occurring when using the website.

main.py

- The “main” backend file used for enabling CORS, GET, POST, and DELETE data using FastAPI. The FastAPI then connects to the MongoDB Atlas and get or receive data from the Atlas.

Frontend

index.html

- The main web page of the project. You can see currently available movies, showtimes and seat grid of each movie, and you can add your information to book a seat. There’s also a hyperlink for the history.html so you can navigate there easily.

script.js

- Used for index.html. Creates functions to lo ad movies, booking, showtimes dropdown, update the seat grid, and validate forms for button, and grabs and sends information to the backend.

history.html

- The second web page used to view your bookings and cancel any bookings made by entering your email address. Also has a hyperlink to send people back to the main web page.

history.js

- Used for history.html. Creates load history and delete booking functions.

style.css

- Used to make the website more movie themed, including dark color background and buttons.

6. Technologies Used

Python

- Used mainly for backend coding, getting, and receiving data through the help of FastAPI.

HTML

- Used for website designs, where to insert user's information, where to navigate to another web page, etc.

JavaScript

- Used to make the website interactive and connect with the backend to receive or send data.

CSS

- Used to styling the website to be more movie themed, coloring buttons, changing background color, and much more.

7. API Endpoints

METHOD	ENDPOINT	DESCRIPTION
GET	/movies	Get registered movies
GET	/booked-seats	Get booked seats
GET	/booking-history	Get user booking history
POST	/users	Create a new user
POST	/book	Create a booking
DELETE	/booking/{booking_id}	Delete a booking

8. Implementation Details

Booking Creation

- The booking feature is implemented using a FastAPI POST endpoint. When the user selects a movie, showtime, and seat on the frontend, the data is sent to the backend using the Fetch API. The backend receives it and stores the data in the MongoDB database.

Booking History

- The booking history is implemented through the /booking-history GET endpoint. The system first retrieves the user document using the provided email. It then queries the *bookings* collection using the user's ID. For each booking, the corresponding movie title is fetched from the *movies* collection. The results are then formatted and returned to the frontend as a JSON response.

Delete Booking

- The delete booking function is implemented using a DELETE endpoint. When the user clicks the "Delete Booking" button, the frontend sends a request containing the booking ID. The backend then converts the ID into an ObjectId and deletes the matching record.

Frontend-Backend Communication

- The frontend communicates with the backend through RESTful API using the Fetch API. Requests are used to send and receive JSON data, making the user interaction smooth without reloading the page.

MongoDB ObjectId Handling

- Since MongoDB uses ObjectId for its documents, the backend converts them into string format first before returning responses, this ensures compatibility with the frontend.

Error Handling

- Error handling is implemented using conditional checks and HTTP exceptions. For example, if a booking is not found during the deletion process, the system returns a 404 error. This improves the system reliability and user feedback.

9. Conclusion

The project implemented the four basic operations (CRUD) successfully and is able to store the data in the MongoDB Atlas with the use of FastAPI. The project also includes both backend and frontend which makes it a full-stack movie ticket booking system project.

In the future, the website might get even more features, updates, and newer designs implemented.